

NanoVDB `ReadAccessor` Benchmark — OLD vs NEW, `Acc=<0,1,2>` vs `Acc=<0>`, CPU vs GPU

Benchmarks comparing the `NANOVDB_USE_OLD_ACCESSOR` caching behaviour (ON vs OFF) and `ReadAccessor<0,1,2>` (full 3-level cache, default) vs `ReadAccessor<0>` (leaf-only cache) across access patterns, on CPU (single- and multi-threaded) and GPU. Results are collected from three machines: a **laptop**, a **desktop**, and a **workstation**. The OLD/NEW comparison for Laptop and Desktop used only `ReadAccessor<0,1,2>`; the full accessor-type comparison was run on the Workstation.

What is being measured

The `ReadAccessor` caches the tree nodes visited on the previous access so a spatially nearby access can skip the traversal from the root.

- **OLD** (`NANOVDB_USE_OLD_ACCESSOR` defined): the `get()` method uses `if constexpr + else` chaining. For `getValue` (operation `LEVEL=0`), the `if constexpr(LEVEL<=0)` branch is taken and the `else` discards all higher-level cache checks — so the 3-level accessor falls straight to a **root traversal** on any leaf-cache miss, effectively behaving like a leaf-only accessor.
- **NEW** (`NANOVDB_NO_OLD_ACCESSOR` defined): the `else` is removed, so all three cache-level checks execute unconditionally. A leaf-cache miss can still hit the **level-1 / level-2** cache instead of restarting at the root.
- **`ReadAccessor<0,1,2>`** (`DefaultReadAccessor`): maintains leaf, lower-internal, and upper-internal node caches. With NEW, the full 3-level check is active. With OLD, the extra levels are stored but never checked on a read — making it functionally equivalent to a larger `ReadAccessor<0>` struct.
- **`ReadAccessor<0>`**: caches only the leaf node. On a leaf miss it always goes straight to root. Unaffected by the OLD/NEW flag (no level-1/2 to check).

Methodology

What the numbers measure

Every reported figure is the **wall-clock time of a single random-access read** into the grid, i.e. one `accessor.getValue(ijk)` call, expressed in **nanoseconds per lookup** (ns/lookup). It is a pure *read-path* micro-benchmark: no values are written, no math is done on the result beyond a running sum (kept `volatile` / written to an output array so the compiler cannot elide the work).

- **CPU-1T** — one thread walks the coordinate list serially with a single accessor. This is a **latency** number: the cost of one dependent lookup.
- **CPU-MT** — the same total work split across all hardware threads (`nanovdb::util::forEach` → `tbb::parallel_for`), each grain using its own accessor. This is a **throughput** number: total time ÷ total lookups with all cores busy.
- **GPU** — the coordinate list is uploaded to the device; each thread processes a 32-coord chunk with its own accessor. Timed with CUDA events (kernel time only, excluding the one-time grid upload). Also a **throughput** number.

The only thing that differs between the OLD and NEW columns is the accessor's cache-fallback logic (a compile-time switch); the data, coordinates, and harness are identical.

The test data

A **fog-volume sphere** built with

```
nanovdb::tools::createFogVolumeSphere<float>(radius=500,
voxelSize=1.0, halfWidth=3.0) . Its measured properties:
```

Property	Value
Value type	float
Index bounding box	[-500, -500, -500] ... [500, 500, 500] (1001 ³ voxels)
Dense voxel count of bbox	1,003,003,001 (~1.00 B)
Active voxels	523,592,077 (~523.6 M)
Fill ratio (active / dense bbox)	52.2 %
Upper internal nodes (level 2, 32 ³)	8
Lower internal nodes (level 1, 16 ³)	272
Leaf nodes (level 0, 8 ³)	82,608
Grid size in memory	188,484,640 bytes (~180 MB)

So it is a **fully populated solid ball** — the interior is all active voxels (the 52 % fill ratio is just the volume of a sphere inside its cube, $\pi/6 \approx 0.524$). It is *sparse* in the VDB sense (only the ball is stored, not the empty corners), but the region the benchmark actually samples is **100 % active**: all access patterns are confined to a cube well inside radius 500, so every lookup lands on a real leaf and traverses the tree for real — we are **not** measuring empty-space shortcuts, we are measuring genuine node traversal and cache reuse.

How much work, how it is averaged

Point patterns (Sequential / LeafJump / NodeJump / Random)	1,048,576 lookups each
Stencil	262,144 centres × 27 neighbours = 7,077,888 lookups
Sampled region	cube of half-extent 256 (points) / [0, 64] ³ dense cube (stencil), both fully inside the ball
Repetition	each configuration run 7 times; the median is reported (robust to OS jitter / turbo ramp)
Warm-up	GPU does one untimed warm-up launch before the 7 timed trials

Hardware / build

Two machines were benchmarked; results for each are in separate sections below.

	Laptop	Desktop	Workstation
CPU	32 HW threads; TBB parallel_for, 4096-coord grains	AMD Ryzen 9 9950X — 16 cores / 32 threads; same TBB settings	AMD Ryzen Threadripper PRO 7975WX — 32 cores / 64

	Laptop	Desktop	Workstation
			threads; same TBB settings
GPU	NVIDIA RTX 5000 Ada Generation Laptop GPU (SM 8.9); 32-coord chunks/thread, 128 threads/block	NVIDIA RTX PRO 6000 Blackwell Max-Q Workstation Edition (SM 12.0); same CUDA settings	NVIDIA RTX 6000 Ada Generation (SM 8.9, 49 GB); same CUDA settings
Build	Release (-O3 -DNDEBUG), C++17 / CUDA 17	same	same
OLD vs NEW	selected at compile time per target (-DNANOVDDB_USE_OLD_ACCESSOR vs -DNANOVDDB_NO_OLD_ACCESSOR)	same	same

Access patterns

Pattern	Stride / shape	Cache behaviour
Sequential	1	leaf (level-0) cache always hot
LeafJump	8 (= leaf dim)	leaf cache cold; level-1 warm (NEW only)
NodeJump	128 (= lower-node dim)	leaf + level-1 cold; level-2 warm (NEW only)
Random	uniform in cube	all caches cold — both fall to root
Stencil	3×3×3 = 27 neighbours per centre, dense sweep	mostly same-leaf hits; boundary neighbours spill to level-1 (NEW rescues)

Results — Desktop (AMD Ryzen 9 9950X / RTX PRO 6000 Blackwell, SM 12.0)

Combined — ns per lookup (one `getValue` call), lower = faster

The stencil row is the per-lookup cost inside a 27-neighbour sweep.

Pattern	CPU-1T OLD	CPU-1T NEW	CPU-MT OLD	CPU-MT NEW	GPU OLD	GPU NEW
Sequential (stride 1)	2.45	1.44	0.26	0.18	0.035	0.020
LeafJump (stride 8)	2.06	1.41	0.21	0.16	0.035	0.020
NodeJump (stride 128)	1.85	1.64	0.22	0.21	0.028	0.024
Random (uniform)	7.19	9.93	0.47	0.57	0.039	0.047
Stencil (3×3×3, 27-pt)	2.143	1.371	0.170	0.102	0.0587	0.0275

Legend: CPU-1T = single thread · CPU-MT = 32 threads (TBB) · GPU = RTX PRO 6000 Blackwell.

OLD → NEW speedup (>1 = the fix is faster)

Pattern	CPU-1T	CPU-MT	GPU
Sequential	1.70×	1.44×	1.75×
LeafJump	1.46×	1.31×	1.75×
NodeJump	1.13×	1.05×	1.17×
Random	0.72×	0.82×	0.83×
Stencil	1.56×	1.67×	2.14×

Detailed tables — Desktop

CPU, single-threaded (latency) — ns per lookup

Pattern	OLD	NEW	OLD → NEW
Sequential	2.45	1.44	1.70×
LeafJump	2.06	1.41	1.46×
NodeJump	1.85	1.64	1.13×
Random	7.19	9.93	0.72×
Stencil	2.143	1.371	1.56×

CPU, 32 threads (throughput) — ns per lookup

Pattern	OLD	NEW	OLD → NEW	MT speedup (NEW)
Sequential	0.26	0.18	1.44×	8.0×
LeafJump	0.21	0.16	1.31×	8.8×
NodeJump	0.22	0.21	1.05×	7.8×
Random	0.47	0.57	0.82×	17.4×
Stencil	0.170	0.102	1.67×	13.4×

GPU (throughput) — ns per lookup

Pattern	OLD	NEW	OLD → NEW
Sequential	0.035	0.020	1.75×
LeafJump	0.035	0.020	1.75×
NodeJump	0.028	0.024	1.17×
Random	0.039	0.047	0.83×
Stencil	0.0587	0.0275	2.14×

Stencil (3×3×3) — reported per whole 27-neighbour stencil

Platform / Mode	OLD ns/stencil	NEW ns/stencil	OLD → NEW
CPU 1 thread	57.86	37.01	1.56×
CPU 32 threads	4.58	2.74	1.67×
GPU	1.5862	0.7413	2.14×

Fair platform comparison — NEW accessor, full hardware (ns per lookup)

Workload	CPU-1T	CPU-MT	GPU	GPU vs CPU-MT
Sequential	1.44	0.18	0.020	9.0×
Random	9.93	0.57	0.047	12.1×

Workload	CPU-1T	CPU-MT	GPU	GPU vs CPU-MT
Stencil	1.371	0.102	0.0275	3.7×

Results — Laptop (RTX 5000 Ada Generation Laptop GPU, SM 8.9)

Combined — ns per lookup (one `getValue` call), lower = faster

The stencil row is the per-lookup cost inside a 27-neighbour sweep.

Pattern	CPU-1T OLD	CPU-1T NEW	CPU-32T OLD	CPU-32T NEW	GPU OLD	GPU NEW
Sequential (stride 1)	3.57	1.96	0.56	0.46	0.058	0.058
LeafJump (stride 8)	3.82	2.21	0.55	0.49	0.057	0.057
NodeJump (stride 128)	3.94	3.05	0.58	0.53	0.054	0.055
Random (uniform)	11.47	15.32	0.82	1.04	0.088	0.095
Stencil (3×3×3, 27-pt)	4.61	2.13	0.322	0.191	0.086	0.040

Legend: CPU-1T = single thread · CPU-32T = 32 threads (TBB) · GPU = RTX 5000 Ada.

OLD → NEW speedup (>1 = the fix is faster)

Pattern	CPU-1T	CPU-32T	GPU
Sequential	1.82×	1.22×	1.00×
LeafJump	1.73×	1.12×	1.00×
NodeJump	1.29×	1.09×	0.98×
Random	0.76×	0.79×	0.93×
Stencil	2.17×	1.68×	2.17×

Detailed tables — Laptop

CPU, single-threaded (latency) — ns per lookup

Pattern	OLD	NEW	OLD → NEW
Sequential	3.57	1.96	1.82×
LeafJump	3.82	2.21	1.73×
NodeJump	3.94	3.05	1.29×
Random	11.47	15.32	0.76×
Stencil	4.61	2.13	2.17×

CPU, 32 threads (throughput) — ns per lookup

Pattern	OLD	NEW	OLD → NEW	MT speedup (NEW)
Sequential	0.56	0.46	1.22×	4.3×
LeafJump	0.55	0.49	1.12×	4.5×

Pattern	OLD	NEW	OLD → NEW	MT speedup (NEW)
NodeJump	0.58	0.53	1.09×	5.7×
Random	0.82	1.04	0.79×	14.7×
Stencil	0.322	0.191	1.68×	11.1×

GPU (throughput) — ns per lookup

Pattern	OLD	NEW	OLD → NEW
Sequential	0.058	0.058	1.00×
LeafJump	0.057	0.057	1.00×
NodeJump	0.054	0.055	0.98×
Random	0.088	0.095	0.93×
Stencil	0.086	0.040	2.17×

Stencil (3×3×3) — reported per whole 27-neighbour stencil

Platform / Mode	OLD ns/stencil	NEW ns/stencil	OLD → NEW
CPU 1 thread	124.48	57.49	2.17×
CPU 32 threads	8.70	5.17	1.68×
GPU	2.32	1.07	2.17×

Fair platform comparison — NEW accessor, full hardware (ns per lookup)

Workload	CPU-1T	CPU-32T	GPU	GPU vs CPU-32T
Sequential	1.96	0.46	0.058	7.9×
Random	15.32	1.04	0.095	10.9×
Stencil	2.13	0.191	0.040	4.8×

Results — Workstation (AMD Ryzen Threadripper PRO 7975WX / RTX 6000 Ada, SM 8.9)

Both `ReadAccessor<0,1,2>` and `ReadAccessor<0>` were benchmarked on this machine under both OLD and NEW compile-time modes. The Laptop and Desktop results (above) used only `ReadAccessor<0,1,2>`; the accessor-type comparison is new here.

CPU single-threaded — ns per lookup (latency)

Pattern	OLD <code>\<0,1,2></code>	OLD <code>\<0></code>	NEW <code>\<0,1,2></code>	NEW <code>\<0></code>
Sequential	3.20	3.12	1.38	2.69
LeafJump	3.27	2.96	1.83	2.75
NodeJump	3.17	2.67	6.65	2.68
Random	8.08	6.81	9.63	6.74
Stencil (ns/lookup)	3.591	2.835	1.587	2.918

CPU 64-threaded — ns per lookup (throughput)

Pattern	OLD <code>\<0,1,2></code>	OLD <code>\<0></code>	NEW <code>\<0,1,2></code>	NEW <code>\<0></code>
Sequential	0.21	0.21	0.22	0.21

Pattern	OLD \<0,1,2>	OLD \<0>	NEW \<0,1,2>	NEW \<0>
LeafJump	0.21	0.23	0.23	0.22
NodeJump	0.22	0.23	0.23	0.19
Random	0.29	0.26	0.39	0.26
Stencil (ns/lookup)	0.137	0.118	0.087	0.123

GPU — ns per lookup (throughput)

Pattern	OLD \<0,1,2>	OLD \<0>	NEW \<0,1,2>	NEW \<0>
Sequential	0.029	0.030	0.024	0.031
LeafJump	0.030	0.029	0.023	0.031
NodeJump	0.025	0.025	0.025	0.026
Random	0.046	0.046	0.055	0.048
Stencil (ns/lookup)	0.0578	0.0577	0.0269	0.0577

OLD → NEW speedup by accessor type

Pattern	CPU-1T \<0,1,2>	CPU-1T \<0>	CPU-MT \<0,1,2>	CPU-MT \<0>	GPU \<0,1,2>	GPU \<0>
Sequential	2.32×	1.16×	0.95×	1.00×	1.21×	0.97×
LeafJump	1.79×	1.08×	0.91×	1.05×	1.30×	0.94×
NodeJump	0.48×	1.00×	0.96×	1.21×	1.00×	0.96×
Random	0.84×	1.01×	0.74×	1.00×	0.84×	0.96×
Stencil	2.26×	0.97×	1.57×	0.96×	2.15×	1.00×

Key: **ReadAccessor<0>** is essentially unchanged by OLD → NEW across all patterns and platforms. The improvement (and regression) from the accessor fix is specific to **ReadAccessor<0,1,2>**.

Stencil detail — ns per whole 27-neighbour stencil

Platform	OLD \<0,1,2>	OLD \<0>	NEW \<0,1,2>	NEW \<0>
CPU 1 thread	96.96	76.56	42.86	78.79
CPU 64 threads	3.71	3.20	2.35	3.31
GPU	1.5597	1.5582	0.7266	1.5576

Best-accessor recommendation (NEW mode, this machine)

Pattern	Best choice	Why
Sequential	<0,1,2> (1.38 ns)	Level-1 cache stays warm between leaf-boundary crossings
LeafJump	<0,1,2> (1.83 ns)	Same — level-1 rescues leaf misses
NodeJump	<0> (2.68 ns)	Level-2 cache checks add overhead; root lookup is cheaper here
Random	<0> (6.74 ns)	All levels miss; extra checks are pure cost
Stencil	<0,1,2> (1.587 ns/lkp)	Boundary neighbours caught by level-1; ~1.84× over <0>

Fair platform comparison — best accessor (NEW), full hardware

Workload	CPU-1T	CPU-64T	GPU	GPU vs CPU-64T
Sequential (<0, 1, 2>)	1.38	0.22	0.024	9.2×
Random (<0>)	6.74	0.26	0.048	5.4×
Stencil (<0, 1, 2>)	1.587	0.087	0.0269	3.2×

Cross-machine comparison (Laptop vs Desktop vs Workstation)

Note: Laptop and Desktop data uses ReadAccessor<0, 1, 2> only. Workstation data covers both accessor types; the cross-machine comparison below uses <0, 1, 2> for consistency, except where accessor type is the explicit subject.

1. The stencil win is universal and consistent for \<0,1,2>

The NEW <0, 1, 2> accessor's stencil speedup is **~2× on GPU and 1.6–2.3× on CPU across all three machines**. It is the most reproducible finding and does not depend on CPU generation, thread count, or GPU model.

ReadAccessor<0> sees **no stencil benefit** from NEW on any platform ($\approx 1.00\times$), confirming the gain comes entirely from the level-1 cache catching boundary neighbours — a feature absent in the leaf-only accessor.

2. NodeJump CPU-1T: NEW \<0,1,2> is dramatically slower on the Workstation

Pattern	WS OLD \<0,1,2>	WS NEW \<0,1,2>	WS NEW \<0>
NodeJump	3.17	6.65	2.68

With NEW + <0, 1, 2>, the NodeJump pattern (stride=128, one lower-internal-node span) triggers a level-2 cache hit after two failed checks (leaf miss + level-1 miss). On this machine and sphere grid (only 8 level-2 nodes), the overhead of two failed cache comparisons plus starting from level-2 exceeds the cost of the root map lookup (8 children), making NEW <0, 1, 2> ~2× slower than both OLD <0, 1, 2> and NEW <0>. ReadAccessor<0> avoids this by falling directly to root after the single leaf check.

Previous runs (Laptop, Desktop) showed NodeJump <0, 1, 2> gains of 1.13–1.29×. The workstation regression may reflect the Threadripper PRO's larger per-core L2 cache making the root path unusually cheap, or a latency difference in how the two paths are branch-predicted.

3. GPU point-pattern speedup is GPU-model-dependent, not architecture alone

ReadAccessor<0, 1, 2> with NEW, OLD → NEW speedup for point patterns:

Pattern	Laptop Ada RTX 5000 (SM 8.9)	Workstation Ada RTX 6000 (SM 8.9)	Desktop Blackwell RTX PRO 6000 (SM 12.0)
Sequential	1.00×	1.21×	1.75×
LeafJump	1.00×	1.30×	1.75×
NodeJump	0.98×	1.00×	1.17×
Random	0.93×	0.84×	0.83×

Both Ada GPUs share SM 8.9 yet differ: RTX 6000 (workstation) extracts 1.21–1.30× for coherent patterns while RTX 5000 (laptop) is neutral. Blackwell (SM 12.0) consistently extracts the most. ReadAccessor<0> with NEW is neutral (≈1.00×) on GPU for all patterns on all machines.

4. GPU Stencil: only \<0,1,2> benefits from NEW

Machine	GPU Stencil OLD \<0,1,2>	GPU Stencil NEW \<0,1,2>	GPU Stencil NEW \<0>
Workstation (Ada RTX 6000)	0.0578	0.0269 (2.15×)	0.0577 (≈1.00×)
Laptop (Ada RTX 5000)	0.086	0.040 (2.17×)	—
Desktop (Blackwell)	0.0587	0.0275 (2.14×)	—

The stencil GPU speedup is ~2.15× for <0,1,2> on all three machines and ~1.00× for <0>. This confirms the level-1 cache is the source of the gain.

5. CPU MT regression is tied to thread count AND accessor type

ReadAccessor<0,1,2> OLD → NEW, multi-threaded:

Pattern	Laptop 32T	Desktop 32T	Workstation 64T
Sequential	1.22×	1.44×	0.95×
NodeJump	1.09×	1.05×	0.96×
Stencil	1.68×	1.67×	1.57×

ReadAccessor<0> OLD → NEW is 1.00× (neutral) for all MT patterns on all machines.

The workstation's MT point-pattern regression is specific to <0,1,2>: the extra cache-level checks add overhead when 64 threads scatter across the grid. With <0>, there are no extra checks so there is no regression.

6. Random always regresses for \<0,1,2>; \<0> is neutral

For <0,1,2>: all caches miss, extra checks are pure overhead → 0.73–0.84× on CPU-1T, 0.83–0.93× on GPU. For <0>: no extra checks → 1.00–1.01× on all platforms.

Summary and bottom line

The NEW fix is unambiguously beneficial for ReadAccessor<0,1,2> on stencil and coherent point patterns. The gains are 1.5–2.3× and portable across CPU and GPU. ReadAccessor<0> (leaf-only) sees no benefit from the fix and no regression — it is effectively unchanged.

For production use with the NEW accessor, the choice of accessor type matters:

Use case	Recommended accessor
Stencil / convolution / dense sweep	ReadAccessor<0,1,2> (NEW gains 2×)
Sequential / LeafJump access	ReadAccessor<0,1,2> (NEW gains 1.4–2.3×)
NodeJump / random scatter	ReadAccessor<0> (avoids regression, same or faster)
Unknown / mixed	ReadAccessor<0,1,2> — wins on the common workloads

Per-machine takeaways

- **Stencil access is the biggest and most universal win** for `ReadAccessor<0,1,2>`: **~2× on GPU** and **1.6–2.3× on CPU** across all three machines. The level-1 cache catches boundary-crossing neighbours that OLD sent to root; the saving compounds across the 27 lookups per centre. `ReadAccessor<0>` sees no stencil benefit on any platform.
- **Coherent point access benefits on single-threaded CPU** with `<0,1,2>` (1.4–2.3× depending on machine). GPU gains are model-dependent: neutral on RTX 5000 Ada (laptop), 1.21–1.30× on RTX 6000 Ada (workstation), 1.75× on Blackwell (SM 12.0).
- **NodeJump CPU-1T with NEW `<0,1,2>` regresses 2× on the Workstation** (6.65 vs 3.17 ns), while `<0>` is neutral (2.67–2.68 ns in both OLD and NEW). When the pattern lands exactly on level-2 cache hits after two failed checks, the overhead exceeds the root-lookup cost for a small grid. `ReadAccessor<0>` avoids this entirely.
- **High thread-count CPU (Threadripper PRO, 64T) shows MT regression** for `<0,1,2>` point patterns (0.70–0.96×): with 64 threads scattering across the grid, the extra cache checks in NEW cost more than they save. `<0>` is neutral in MT too. The stencil win holds for `<0,1,2>` at 1.57× even on 64 threads.
- **`ReadAccessor<0>` is effectively unchanged by the OLD → NEW flag** on all platforms, patterns, and thread counts. Its code path (leaf check → root) is unaffected by the `#ifdef`-controlled `else` blocks that only appear in `<0,1,2>`.
- **The multi-threaded CPU makes the GPU comparison fair.** Against a fully loaded 64-thread Threadripper PRO the GPU (RTX 6000 Ada) is ~3–9× faster (not the 50–280× a single thread would suggest). The gap is smallest for stencil (~3×), reflecting the CPU's larger per-core caches absorbing the coherent reuse.
- **CPU multi-thread scaling:** on the 64-thread Threadripper PRO, stencil `<0,1,2>` NEW achieves ~18× scaling (1T → 64T), random achieves ~28× (latency-hiding), and sequential/coherent patterns hit ~6–7× (memory-bandwidth-limited).

Building & running

```
# Configure with the benchmark (and CUDA, for the GPU variant) enabled
cmake -S . -B build -DNANOVDB_BUILD_BENCHMARK=ON -DNANOVDB_USE_CUDA=ON

# Build all four executables
cmake --build build --target \
  bench_accessor_old bench_accessor_new \
  bench_accessor_cuda_old bench_accessor_cuda_new -j

# Run – each binary prints results for both ReadAccessor<0,1,2> and ReadAccessor<0>
cd build/nanovdb/nanovdb/benchmark
./bench_accessor_old      # CPU, OLD accessor, both <0,1,2> and <0>
./bench_accessor_new      # CPU, NEW accessor, both <0,1,2> and <0>
./bench_accessor_cuda_old # GPU, OLD accessor, both <0,1,2> and <0>
./bench_accessor_cuda_new # GPU, NEW accessor, both <0,1,2> and <0>
```

The OLD vs NEW behaviour is selected at compile time per target (-DNANOVDDB_USE_OLD_ACCESSOR vs -DNANOVDDB_NO_OLD_ACCESSOR). Both accessor types (ReadAccessor<0,1,2> and ReadAccessor<0>) are run within each binary.

Files

File	Purpose
BenchPatterns.h	Shared access-pattern generation (identical coords for CPU & GPU)
BenchAccessor.cc	CPU benchmark (single- and multi-threaded)
BenchAccessorCuda.cu	GPU (CUDA) benchmark
CMakeLists.txt	Builds the OLD/NEW × CPU/GPU targets